Tugas Pertemuan 1	
IF21W0508 – Praktikum Web	
NPM	243040039
Nama	Raja Saut Sihaloho
Kelas	F
Link Github Repository	https://github.com/Rajasaut/PW2026_243040039/tree/main/pertemuan2

Bagian 1. Screenshot Semua Kode

1. main.tsx

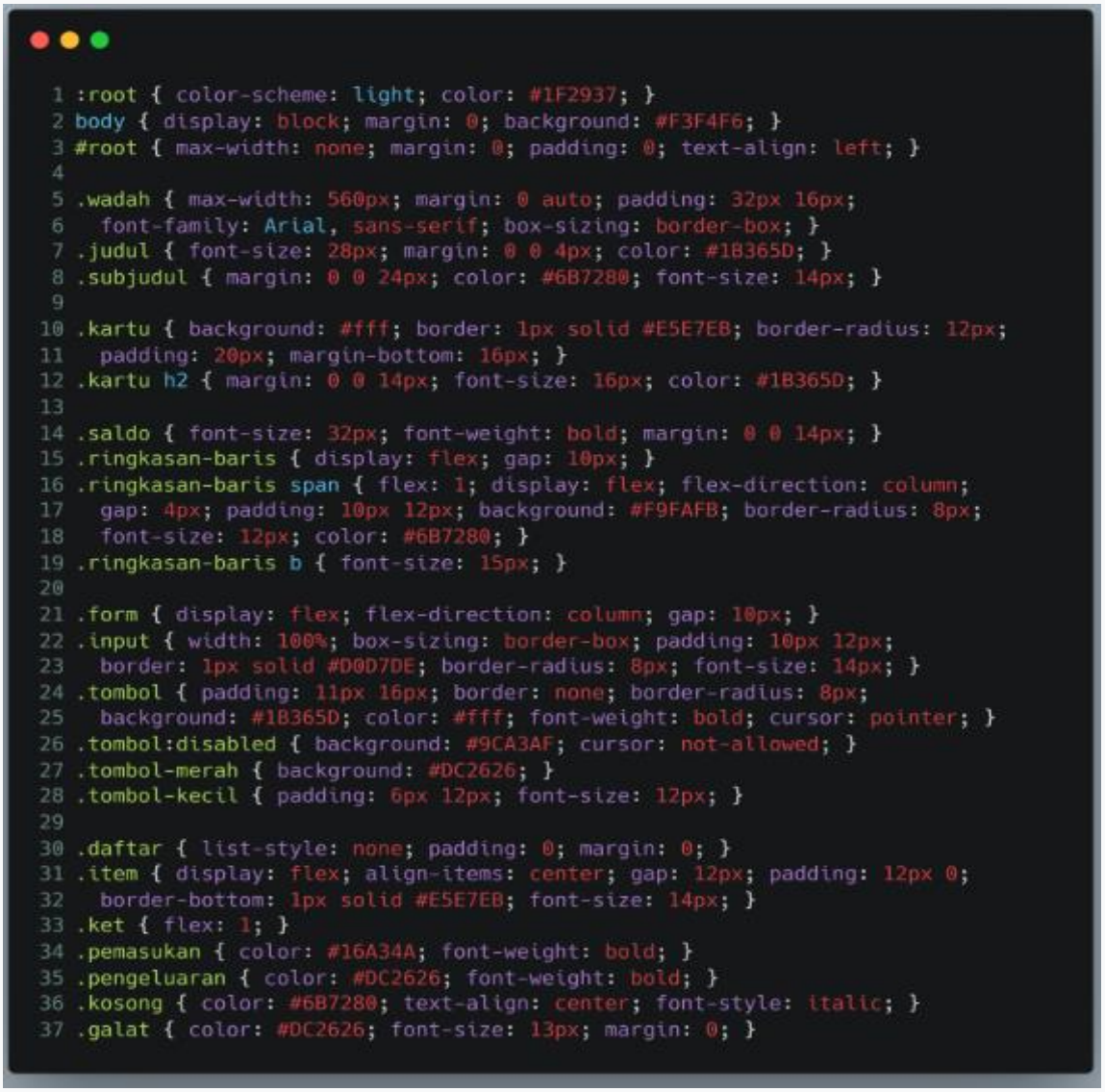


```tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import CatatanKeuangan from "./Tugas2/CatatanKeuangan";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <CatatanKeuangan />
  </StrictMode>,
);

```


2. index.css



```css
:root { color-scheme: light; color: #1F2937; }
body { display: block; margin: 0; background: #F3F4F6; }
#root { max-width: none; margin: 0; padding: 0; text-align: left; }

.wadah { max-width: 560px; margin: 0 auto; padding: 32px 16px;
  font-family: Arial, sans-serif; box-sizing: border-box; }
.judul { font-size: 28px; margin: 0 0 4px; color: #1B365D; }
.subjudul { margin: 0 0 24px; color: #6B7280; font-size: 14px; }

.kartu { background: #fff; border: 1px solid #E5E7EB; border-radius: 12px;
  padding: 20px; margin-bottom: 16px; }
.kartu h2 { margin: 0 0 14px; font-size: 16px; color: #1B365D; }

.saldo { font-size: 32px; font-weight: bold; margin: 0 0 14px; }
.ringkasan-baris { display: flex; gap: 10px; }
.ringkasan-baris span { flex: 1; display: flex; flex-direction: column;
  gap: 4px; padding: 10px 12px; background: #F9FAFB; border-radius: 8px;
  font-size: 12px; color: #6B7280; }
.ringkasan-baris b { font-size: 15px; }

.form { display: flex; flex-direction: column; gap: 10px; }
.input { width: 100%; box-sizing: border-box; padding: 10px 12px;
  border: 1px solid #D0D7DE; border-radius: 8px; font-size: 14px; }
.tombol { padding: 11px 16px; border: none; border-radius: 8px;
  background: #1B365D; color: #fff; font-weight: bold; cursor: pointer; }
.tombol:disabled { background: #9CA3AF; cursor: not-allowed; }
.tombol-merah { background: #DC2626; }
.tombol-kecil { padding: 6px 12px; font-size: 12px; }

.daftar { list-style: none; padding: 0; margin: 0; }
.item { display: flex; align-items: center; gap: 12px; padding: 12px 0;
  border-bottom: 1px solid #E5E7EB; font-size: 14px; }
.ket { flex: 1; }
.pemasukan { color: #16A34A; font-weight: bold; }
.pengeluaran { color: #DC2626; font-weight: bold; }
.kosong { color: #6B7280; text-align: center; font-style: italic; }
.galat { color: #DC2626; font-size: 13px; margin: 0; }
```

3. Tugas2/helper.tsx

```tsx
export interface Transaksi {
  id: number;
  keterangan: string;
  nominal: number;
  jenis: "pemasukan" | "pengeluaran";
}

export const formatRupiah = (angka: number): string =>
  new Intl.NumberFormat("id-ID", {
    style: "currency",
    currency: "IDR",
    maximumFractionDigits: 0,
  }).format(angka);

```

4. Tugas2/Kartu.tsx

```tsx
import type { ReactNode } from "react";

interface KartuProps {
  judul: string;
  children: ReactNode;
}

function Kartu({ judul, children }: KartuProps) {
  return (
    <section className="kartu">
      <h2>{judul}</h2>
      {children}
    </section>
  );
}

export default Kartu;

```

5. Tugas2/FormTransaksi.tsx

```tsx
import { useState } from "react";
import type { FormEvent } from "react";
import type { Transaksi } from "./helper";

interface FormTransaksiProps {
  onTambah: (
    keterangan: string,
    nominal: number,
    jenis: Transaksi["jenis"],
  ) => void;
}

function FormTransaksi({ onTambah }: FormTransaksiProps) {
  const [keterangan, setKeterangan] = useState("");
  const [nominal, setNominal] = useState("");
  const [jenis, setJenis] = useState<Transaksi["jenis"]>("pemasukan");

  // Input selalu teks, ubah ke angka dengan Number()
  const angka = Number(nominal);
  const tidakValid =
    keterangan.trim() === "" || nominal === "" || isNaN(angka) || angka <= 0;

  const tanganiKirim = (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    if (tidakValid) return;
    onTambah(keterangan.trim(), angka, jenis);
    setKeterangan("");
    setNominal("");
    setJenis("pemasukan");
  };

  return (
    <form className="form" onSubmit={tanganiKirim}>
      <input
        className="input"
        type="text"
        placeholder="Keterangan, contoh: Uang saku"
        value={keterangan}
        onChange={(e) => setKeterangan(e.target.value)}
      />
      <input
        className="input"
        type="number"
        placeholder="Nominal, contoh: 50000"
        value={nominal}
        onChange={(e) => setNominal(e.target.value)}
      />
      <select
        className="input"
        value={jenis}
        onChange={(e) => setJenis(e.target.value as Transaksi["jenis"])}
      >
        <option value="pemasukan">Pemasukan</option>
        <option value="pengeluaran">Pengeluaran</option>
      </select>
      {nominal !== "" && angka <= 0 && (
        <p className="galat">Nominal harus lebih dari 0.</p>
      )}
      <button className="tombol" type="submit" disabled={tidakValid}>
        Tambah Transaksi
      </button>
    </form>
  );
}

export default FormTransaksi;
```

6. Tugas2/ItemTransaksi.tsx

```tsx
import { formatRupiah } from "./helper";
import type { Transaksi } from "./helper";

interface ItemTransaksiProps {
  transaksi: Transaksi;
  onHapus: (id: number) => void;
}

function ItemTransaksi({ transaksi, onHapus }: ItemTransaksiProps) {
  const masuk = transaksi.jenis === "pemasukan";

  return (
    <li className="item">
      <span className="ket">{transaksi.keterangan}</span>
      {/* className dinamis: hijau untuk pemasukan, merah untuk pengeluaran */}
      <span className={masuk ? "pemasukan" : "pengeluaran"}>
        {masuk ? "+" : "-"} {formatRupiah(transaksi.nominal)}
      </span>
      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(transaksi.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTransaksi;
```

7. Tugas2/Ringkasan.tsx

```tsx
import { formatRupiah } from "./helper";
import type { Transaksi } from "./helper";

interface RingkasanProps {
  daftar: Transaksi[];
}

function Ringkasan({ daftar }: RingkasanProps) {
  // Nilai turunan: dihitung dari state, tidak perlu state baru
  const totalMasuk = daftar
    .filter((t) => t.jenis === "pemasukan")
    .reduce((jumlah, t) => jumlah + t.nominal, 0);
  const totalKeluar = daftar
    .filter((t) => t.jenis === "pengeluaran")
    .reduce((jumlah, t) => jumlah + t.nominal, 0);
  const saldo = totalMasuk - totalKeluar;

  return (
    <>
      <p className={`saldo ${saldo >= 0 ? "pemasukan" : "pengeluaran"}`}>
        {formatRupiah(saldo)}
      </p>
      <div className="ringkasan-baris">
        <span>
          Pemasukan: <b className="pemasukan">{formatRupiah(totalMasuk)}</b>
        </span>
        <span>
          Pengeluaran:{" "}
          <b className="pengeluaran">{formatRupiah(totalKeluar)}</b>
        </span>
      </div>
    </>
  );
}

export default Ringkasan;
```

8. Tugas2/CatatanKeuangan.tsx

```tsx
import { useState, useEffect } from "react";
import Kartu from "./Kartu";
import FormTransaksi from "./FormTransaksi";
import ItemTransaksi from "./ItemTransaksi";
import Ringkasan from "./Ringkasan";
import type { Transaksi } from "./helper";

function CatatanKeuangan() {
  // State utama: baca dari localStorage dulu, kalau kosong/rusak pakai array kosong
  const [daftar, setDaftar] = useState<Transaksi[]>(() => {
    try {
      const tersimpan = localStorage.getItem("catatan-keuangan");
      return tersimpan ? JSON.parse(tersimpan) : [];
    } catch {
      return [];
    }
  });

  // Efek: simpan ke localStorage setiap daftar berubah
  useEffect(() => {
    localStorage.setItem("catatan-keuangan", JSON.stringify(daftar));
  }, [daftar]);

  // Efek: judul tab menampilkan jumlah transaksi
  useEffect(() => {
    document.title = `Catatan Keuangan (${daftar.length} transaksi)`;
  }, [daftar]);

  const tambahTransaksi = (
    keterangan: string,
    nominal: number,
    jenis: Transaksi["jenis"],
  ) => {
    const baru: Transaksi = { id: Date.now(), keterangan, nominal, jenis };
    setDaftar([...daftar, baru]);
  };

  const hapusTransaksi = (id: number) => {
    setDaftar(daftar.filter((t) => t.id !== id));
  };

  return (
    <div classNameName="wadah">
      <h1 classNameName="judul">Catatan Keuangan Mini</h1>
      <p classNameName="subjudul">Praktikum Web - Modul 2</p>
      <Kartu judul="Total Saldo">
        <Ringkasan daftar={daftar} />
      </Kartu>
      <Kartu judul="Tambah Transaksi">
        <FormTransaksi onTambah={tambahTransaksi} />
      </Kartu>
      <Kartu judul="Daftar Transaksi">
        {daftar.length === 0 ? (
          <p classNameName="kosong">
            Belum ada transaksi. Yuk catat transaksi pertamamu!
          </p>
        ) : (
          <ul classNameName="daftar">
            {daftar.map((t) => (
              <ItemTransaksi
                key={t.id}
                transaksi={t}
                onHapus={hapusTransaksi}
              />
            ))}
          </ul>
        )}
      </Kartu>
    </div>
  );
}
export default CatatanKeuangan;
```

Bagian 2. Penjelasan Kode

Aplikasi Catatan Keuangan Mini dibangun dari lima komponen dan satu berkas pendukung yang disimpan di folder src/Tugas2. State utama hanya dipegang oleh komponen induk (CatatanKeuangan). Komponen anak hanya menerima data dan fungsi lewat props, sehingga data turun dari induk ke anak dan laporan naik dari anak ke induk (lifting state up).

1. main.tsx

Titik masuk aplikasi. Berkas ini memanggil createRoot, lalu merender komponen CatatanKeuangan di dalam StrictMode. index.css diimpor di sini agar gayanya berlaku untuk seluruh halaman.

2. index.css

Satu-satunya berkas CSS. Bagian atasnya menimpa gaya bawaan Vite (isi yang rata tengah dan bertema gelap) supaya tampilan aplikasi rapi. Setelah itu didefinisikan kelas .wadah, .kartu, .form, .input, .tombol, .item, .pemasukan, .pengeluaran, dan lainnya. Memakai className membuat kode JSX lebih ringkas dibanding inline style.

3. helper.tsx

Berisi interface Transaksi (id, keterangan, nominal, jenis) sebagai cetakan bentuk data, dan fungsi formatRupiah yang memakai Intl.NumberFormat agar angka tampil sebagai Rp 50.000. Keduanya dipisah ke berkas sendiri supaya bisa dipakai bersama oleh beberapa komponen tanpa ditulis ulang.

4. Kartu.tsx

Komponen pembungkus yang menerima props judul dan children. Isi apa pun yang diletakkan di antara tag pembuka dan penutup Kartu akan tampil di dalam bingkai. Dengan cara ini bingkai yang sama dipakai tiga kali (Total Saldo, Tambah Transaksi, Daftar Transaksi) tanpa menulis ulang kodenya.

5. FormTransaksi.tsx

Formulir terkendali (controlled form) dengan tiga state lokal: keterangan, nominal, dan jenis. Setiap input memiliki value yang dibaca dari state dan onChange yang menulis ke state, sehingga isi input selalu sama dengan state. Nilai input selalu berupa teks, jadi nominal diubah dengan Number(nominal) sebelum dikirim. Tanpa itu, penjumlahan 5000 dan 2000 akan menjadi teks "50002000", bukan 7000.

Variabel tidakValid bernilai benar jika keterangan kosong, nominal kosong, atau nominal tidak lebih dari 0. Nilai ini menonaktifkan tombol Tambah dan memunculkan pesan galat lewat operator &&. Saat formulir dikirim, event.preventDefault() mencegah halaman reload, lalu fungsi onTambah dari induk dipanggil dan input dikosongkan. Formulir tidak menyimpan daftar transaksi; ia hanya melapor ke induk.

6. ItemTransaksi.tsx

Menampilkan satu baris transaksi. Komponen ini menerima satu objek transaksi dan fungsi onHapus dari induk. className dinamis dengan ternary memberi warna hijau dan tanda + untuk pemasukan, serta merah dan tanda - untuk pengeluaran. Tombol Hapus ditulis onClick={() => onHapus(transaksi.id)} dengan arrow function karena id

perlu dikirim. Jika ditulis onHapus(transaksi.id) langsung, fungsi akan berjalan saat render dan semua data terhapus sendiri.

7. Ringkasan.tsx

Menghitung total pemasukan, total pengeluaran, dan saldo dari array daftar memakai .filter() dan .reduce(). Ketiganya adalah nilai turunan: dihitung ulang setiap render dan tidak disimpan sebagai state baru. Karena itu angkanya selalu sinkron dengan daftar dan tidak ada state berlebih. Saldo berwarna hijau jika nol atau lebih, dan merah jika negatif.

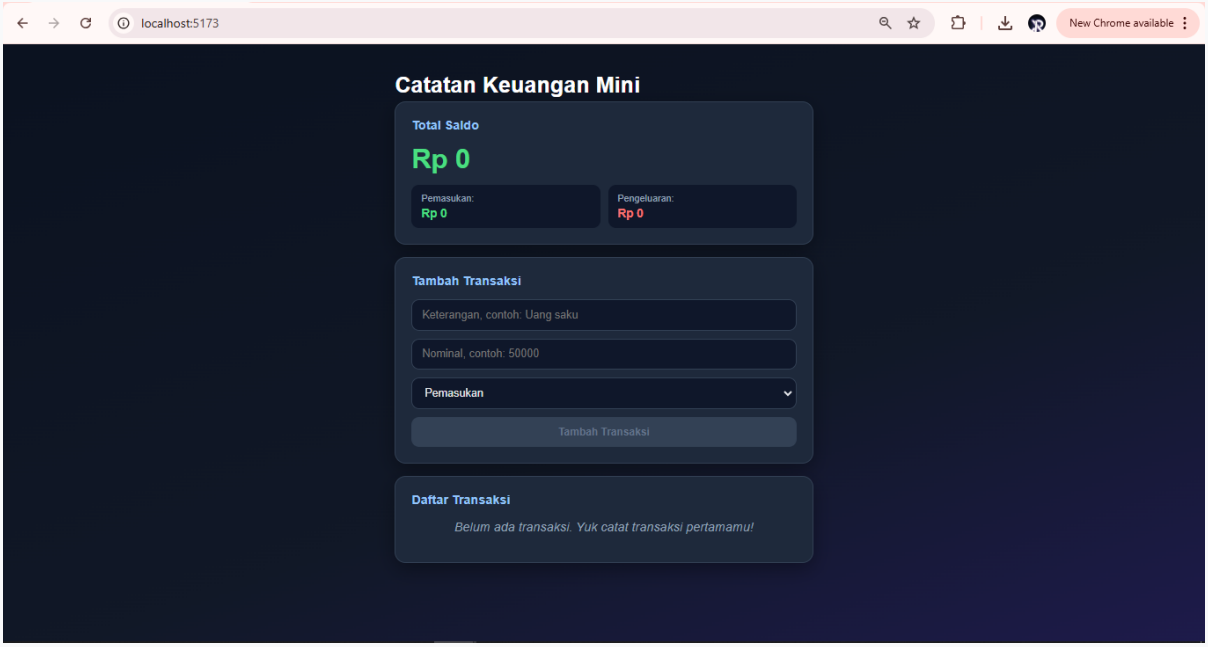
8. CatatanKeuangan.tsx (komponen induk)

Satu-satunya pemilik state daftar. State diinisialisasi dengan fungsi yang membaca localStorage dan JSON.parse. Fungsi ini hanya berjalan sekali saat aplikasi pertama kali tampil, dan dibungkus try/catch supaya data yang rusak tidak membuat aplikasi error. Fungsi tambahTransaksi membuat array baru dengan spread, setDaftar([...daftar, baru]), sedangkan hapusTransaksi memakai .filter(). Keduanya tidak mengubah array lama secara langsung (immutable update), sehingga React dapat mendeteksi perubahan.

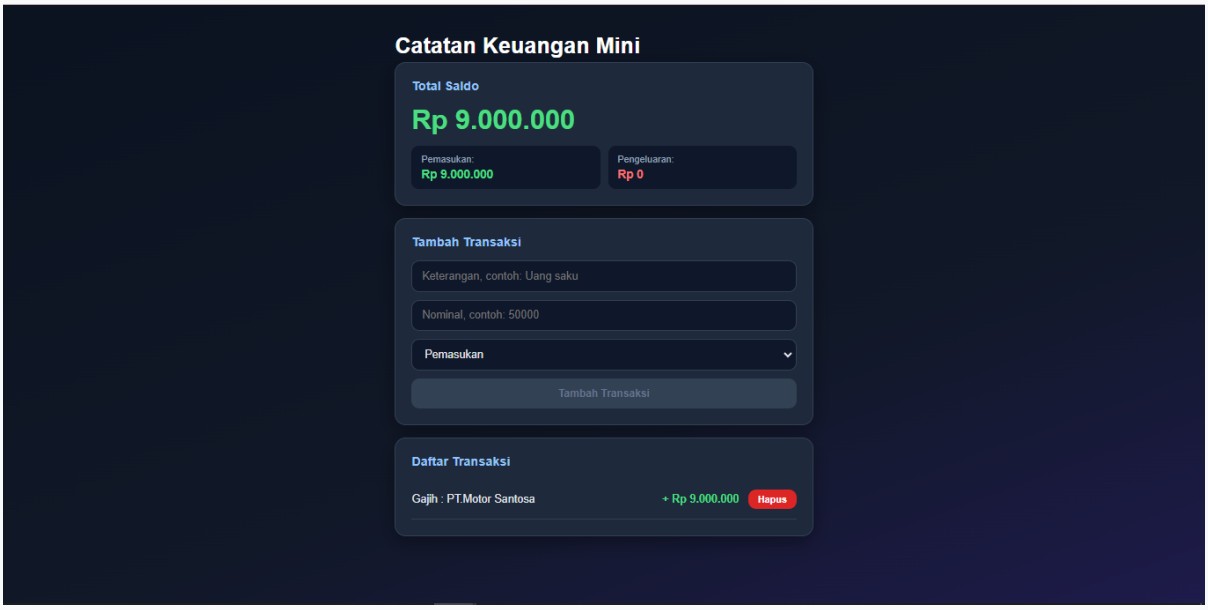
Terdapat dua useEffect dengan dependency array [daftar]. Efek pertama menyimpan daftar ke localStorage dengan JSON.stringify. Efek kedua memperbarui judul tab browser sesuai jumlah transaksi. Dependency array [daftar] membuat efek hanya berjalan saat pertama tampil dan ketika daftar berubah. Pada bagian tampilan, ternary memilih antara pesan "Belum ada transaksi" dan daftar yang dibuat dengan .map() menggunakan key={t.id}.

Bagian 3. Screenshot Hasil Antarmuka

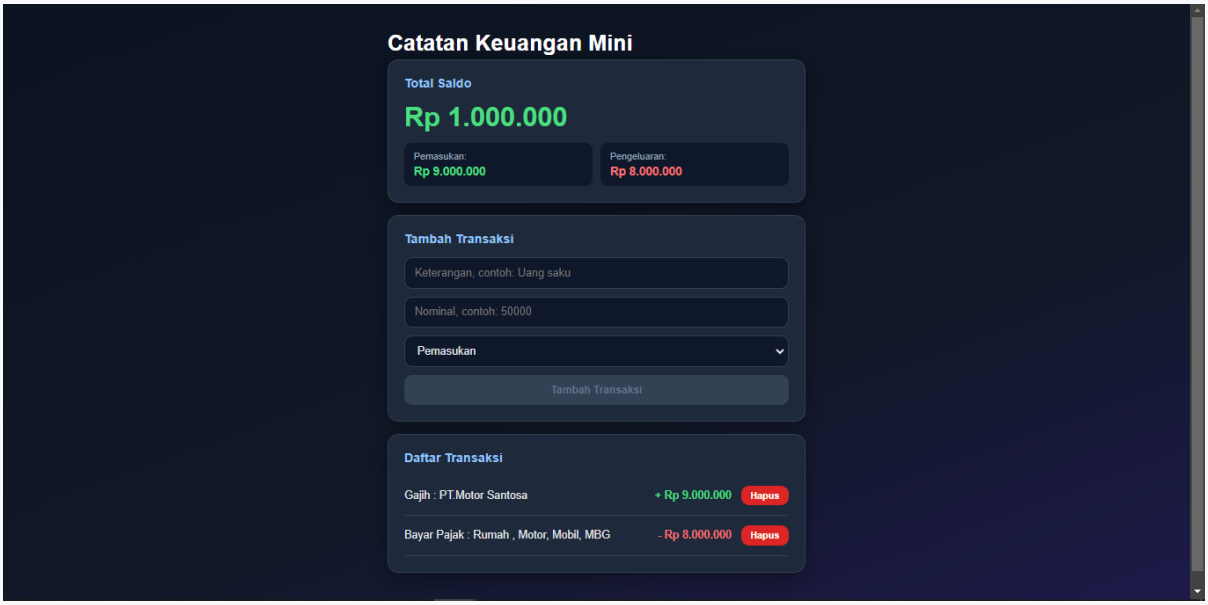
Skenario 1. Kondisi awal (belum ada transaksi)



Skenario 2. Menambah transaksi pemasukan



Skenario 3. Menambah transaksi pengeluaran

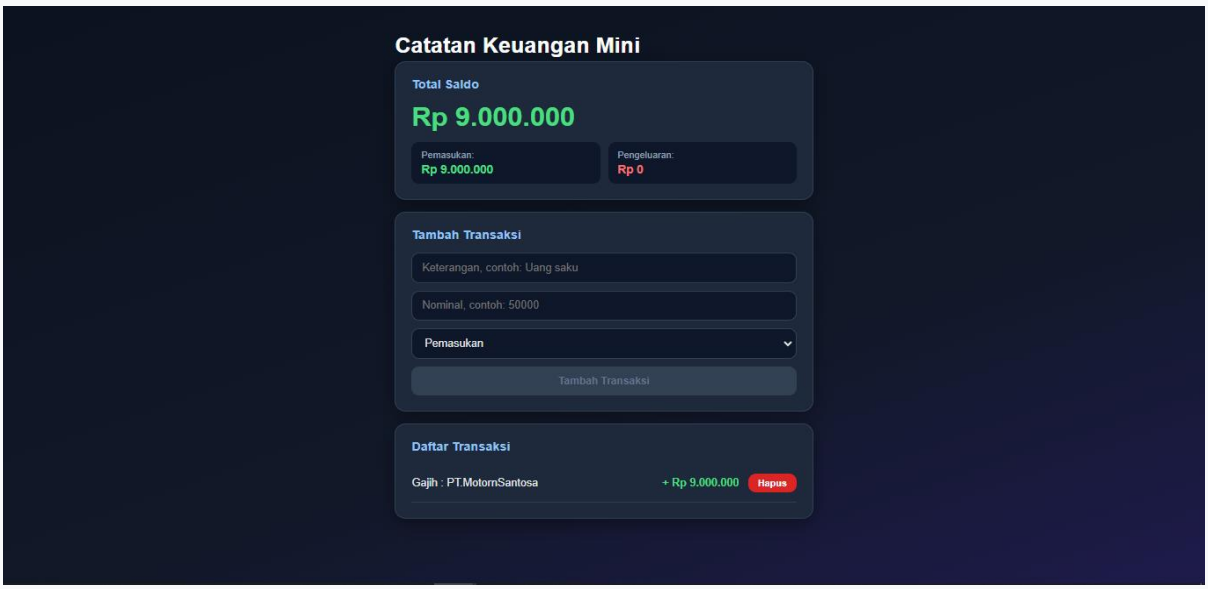


Skenario 4. Validasi input (tombol Tambah nonaktif)



Skenario 5. Menghapus transaksi

Skenario 6. Refresh halaman (F5), data tetap ada



Bagian 4. Penjelasan Hasil Antarmuka

Skenario 1. Kondisi awal

Saldo tampil Rp 0, pemasukan dan pengeluaran Rp 0, serta muncul pesan "Belum ada transaksi". Judul tab menjadi "Catatan Keuangan (0 transaksi)". Hal ini terjadi karena state daftar berupa array kosong, sehingga ternary memilih cabang pesan kosong, dan useEffect memperbarui judul tab.

Skenario 2. Menambah pemasukan

Setelah keterangan dan nominal diisi dengan jenis Pemasukan lalu tombol Tambah ditekan, transaksi muncul di daftar dengan warna hijau dan tanda +. Saldo bertambah, judul tab berubah, dan input kembali kosong. Penyebabnya: FormTransaksi memanggil onTambah, induk membuat array baru dengan spread, lalu React menggambar ulang. Ringkasan menghitung ulang saldo dari array yang baru.

Skenario 3. Menambah pengeluaran

Transaksi pengeluaran tampil berwarna merah dengan tanda - dan saldo berkurang. Jika total pengeluaran melebihi pemasukan, saldo menjadi negatif dan berwarna merah. Warna ditentukan oleh className dinamis berdasarkan nilai jenis, sedangkan saldo dihitung sebagai total pemasukan dikurangi total pengeluaran.

Skenario 4. Validasi input

Ketika keterangan atau nominal kosong, tombol Tambah tampil abu-abu dan tidak dapat diklik. Jika nominal 0 atau negatif, muncul pesan galat berwarna merah. Hal ini terjadi karena formulir bersifat terkendali: nilai input tersimpan di state, sehingga tidakValid dapat dihitung langsung dari state pada setiap ketikan.

Skenario 5. Menghapus transaksi

Saat tombol Hapus diklik, baris tersebut hilang dan saldo ikut menyesuaikan. Jika semua transaksi dihapus, pesan "Belum ada transaksi" muncul kembali. Hal ini karena .filter() menghasilkan array baru tanpa transaksi yang id-nya sama, sehingga React mendeteksi perubahan dan menggambar ulang tampilan.

Skenario 6. Refresh halaman (F5)

Setelah halaman di-refresh, seluruh transaksi dan saldo tetap ada. Alasannya, setiap kali daftar berubah, useEffect menyimpannya ke localStorage. Saat aplikasi dimuat ulang, state awal dibaca kembali dari localStorage. Tanpa useEffect, data hanya hidup di memori dan akan hilang ketika halaman dimuat ulang.